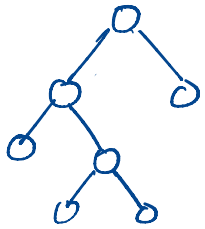
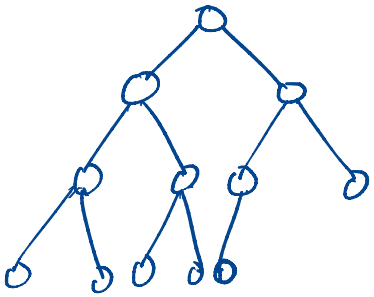
AVL Trees

Full vs. Complete binary tree

→ A full binary tree is one in which all nodes have either two children or none.



→ In a complete binary tree every level except possibly the last one is completely filled and all nodes in the last level are filled from the left



→ In a balanced tree, left & right subtrees of every node differ in height by no more than 1.

# of edges from topmost node (root of the subtree) to the farthest node in the subtree

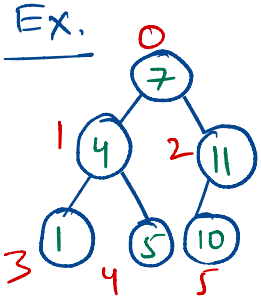
Storing binary trees in arrays

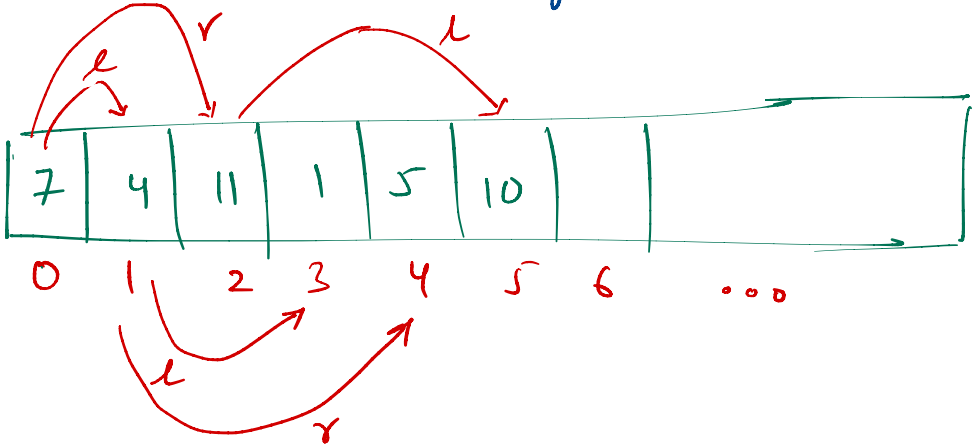
Binary trees can be stored in breadth-first order as an implicit data structure in arrays, and if the tree is a complete binary tree then this method wastes no space.

→ If a node in the binary has an array index (0-based) of i then its children are found at indices $(2i+1)$ for left child, and $(2i+2)$ for right child, while its parent (if any) is found at index $\left\lfloor \frac{i-1}{2} \right\rfloor$, assuming root has index of 0.

→ This gives a compact storage with better locality of reference, however it is expensive to grow and wastes space proportional to $(2^{(h+1)} - n)$ for a tree of height h and n nodes

Ex.





AVL tree: An AVL tree is a BST with Properties:

1. The subtrees of every node differ in height by at most 1.
2. Every subtree is an AVL tree.

Balance Factor of a node x is defined as the height difference:

$$BF(x) = Height(x.right) - Height(x.left)$$

of the two child subtrees rooted at x.

Note: Height of a NIL node is defined to be **-1**

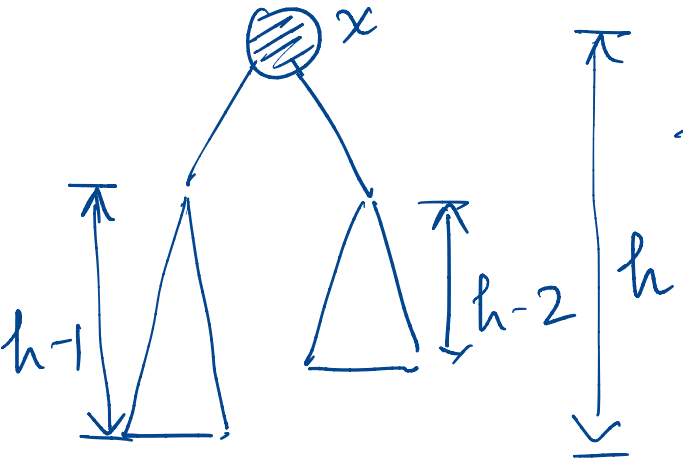
— The goal in AVL trees is to maintain logarithmic height

↳ i.e., $h = lg_2 n$, n = # of nodes

↳ We need to somehow balance the given BST after insertion/deletion

→ Solution proposed by G. M. Adelson-Velskii and E. M. Landis in 1962

Average search takes $1.44 \lg_2 n$ comparisons



→ Balanced at node x

BF: $(h-2) - (h-1) = -1$

→ The invariant for an AVL tree is that at any node x:

$$BF(x) \in \{-1, 0, 1\}$$

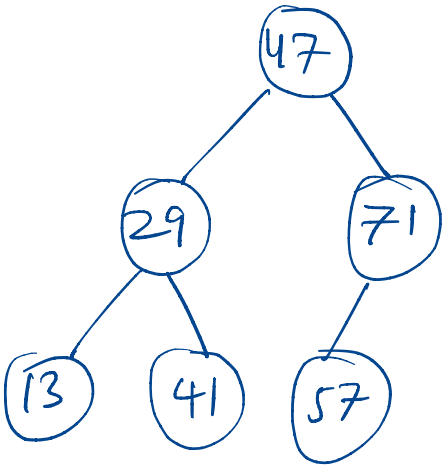
→ An AVL tree node is called:

Left heavy: if $BF(x) < 0$

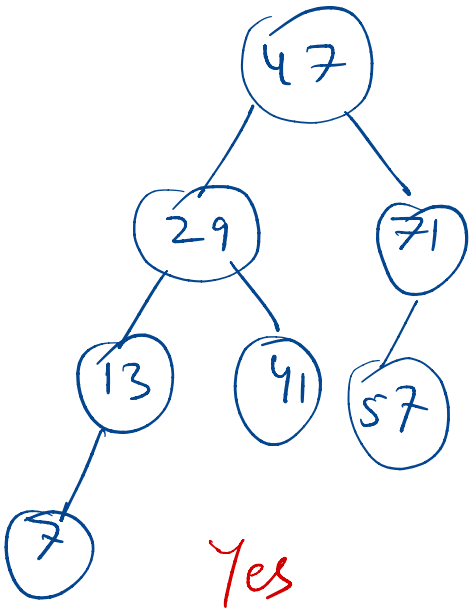
Right heavy: if $BF(x) > 0$

(exactly) Balanced node: if $BF(x) = 0$

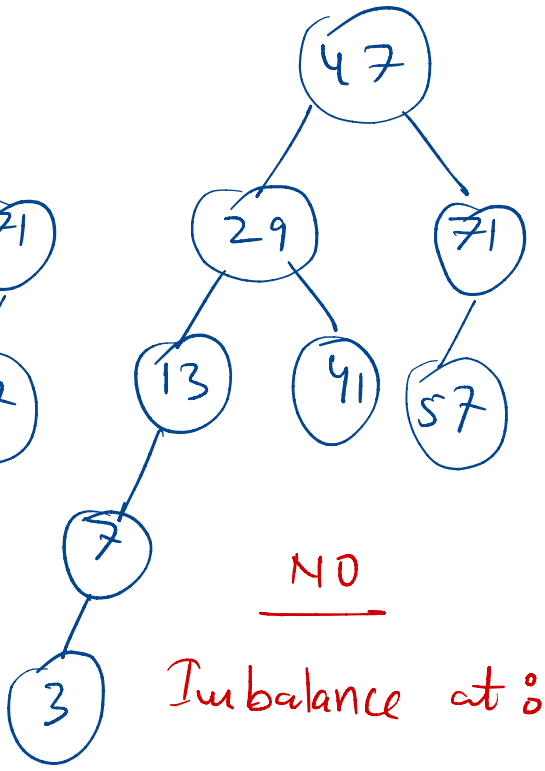
Exaupleo:



Yes



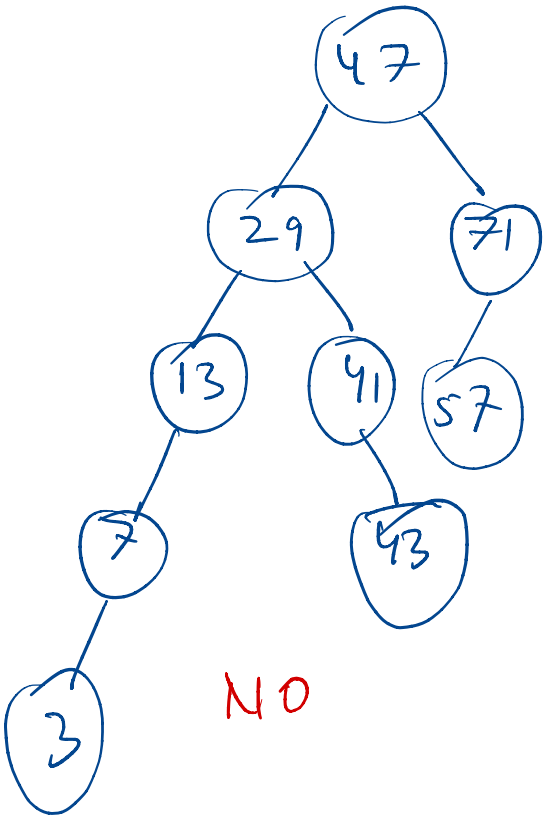
Yes



NO

Imbalance at:

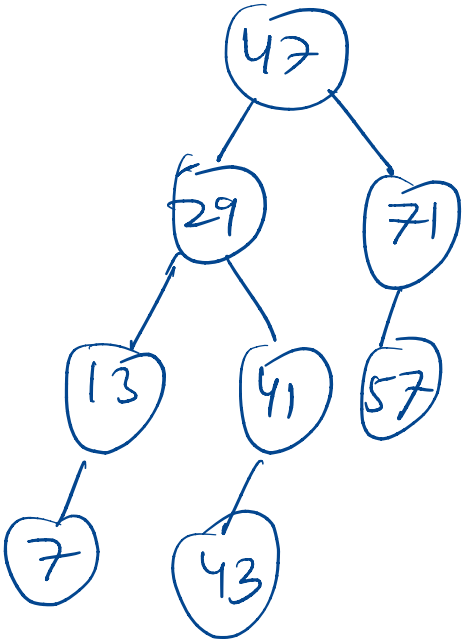
13, 29, 47



NO

Imbalance at:

13, 47

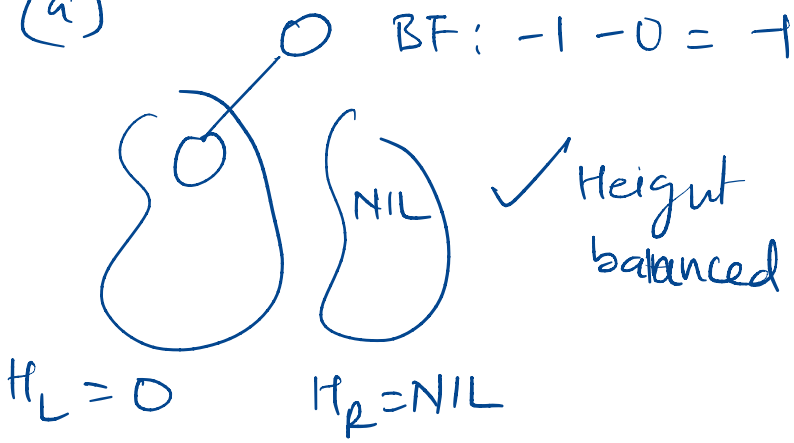


NO

(Not a BST!)

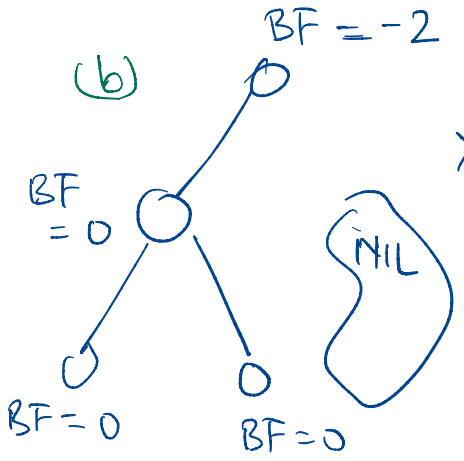
Examples of balance factors:

(a)



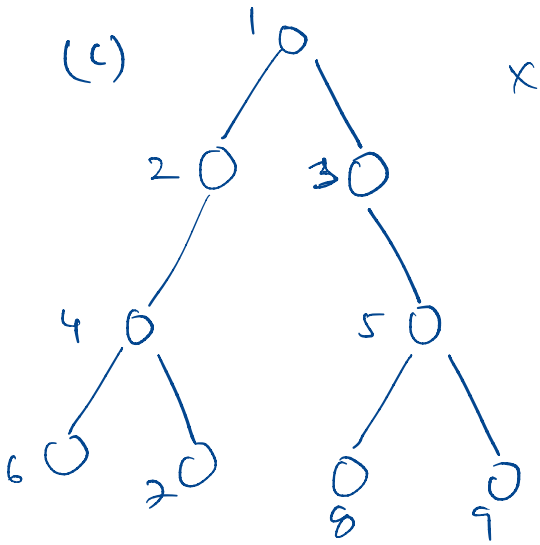
✓ Height balanced

(b)



X Not height balanced

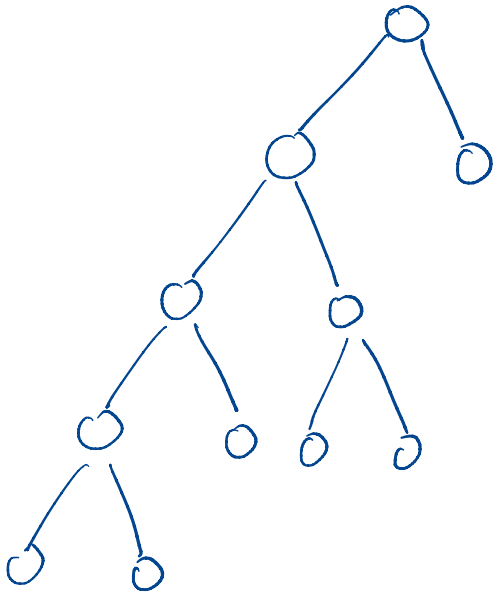
(c)



X Not height balanced.

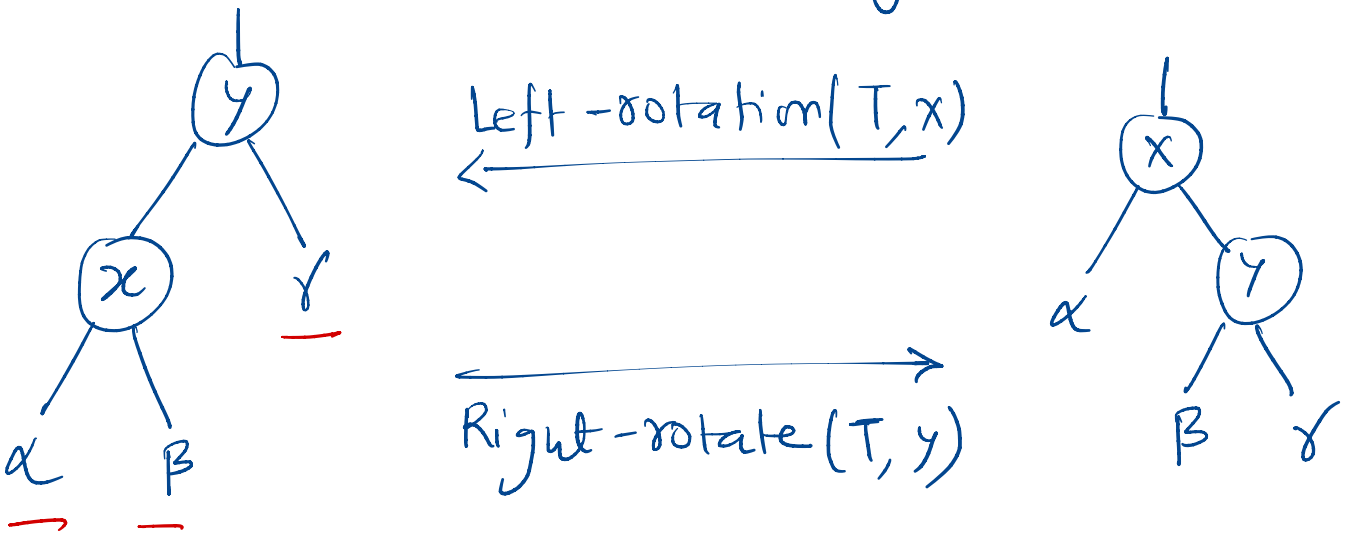
Imbalance at nodes 2 & 3

(d) A full binary tree that's not balanced



Rotation: A local operation that preserves the binary-search-tree property

- ↳ left-rotation
- ↳ right-rotation



arbitrary subtrees

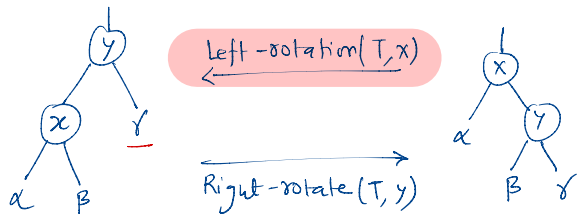
A rotation operation preserves the BST property: the keys in α precede $x.key$, which precedes the keys in β, which precede $y.key$, which precedes keys in γ.

$\Rightarrow keys(\alpha) \leq x.key \leq keys(\beta) \leq y.key \leq keys(\gamma)$

① ② ③ ④

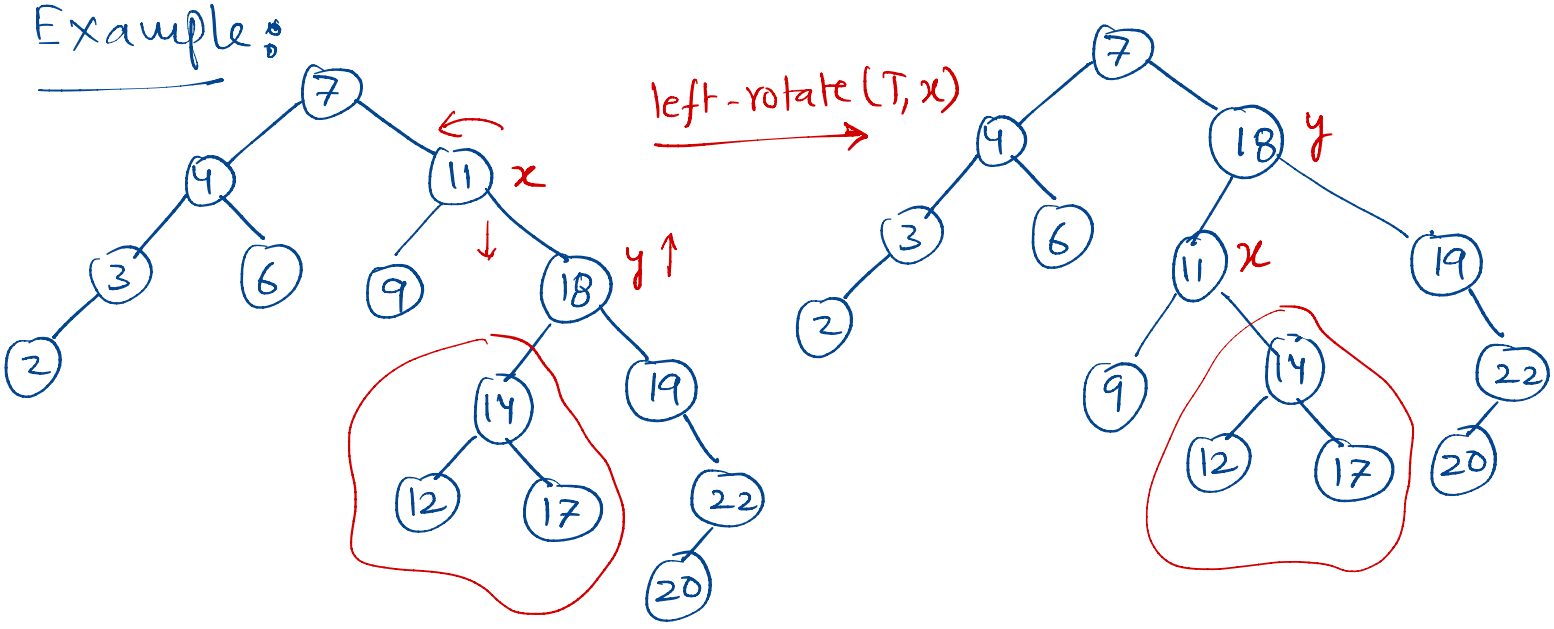
are true! $\Rightarrow$ valid BST

Left-Rotate(T, x)




```
y = x.right  // Assumption: y ≠ NIL
x.right = y.left  // Turn y's left subtree into x's right subtree
if y.left ≠ NIL
        y.left.p = x
y.p = x.p
if x.p == NIL
        T.root = y
else if x == x.p.left
        x.p.left = y
else
        x.p.right = y
y.left = x
x.p = y
```

Example:



AVL tree balancing:

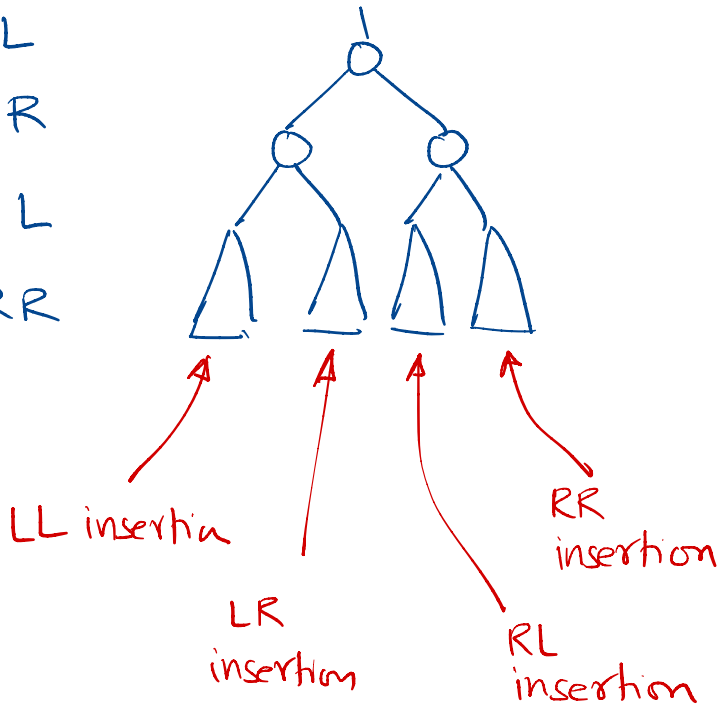
- Insert normally as in a BST
- If the tree becomes imbalanced, then fix it
  - If a tree becomes imbalanced after an insert then only the ancestors of the newly inserted node are imbalanced

(which is the search path nodes encountered while inserting the given node)

To fix the imbalance, we need to focus on the deepest node first that is imbalanced.

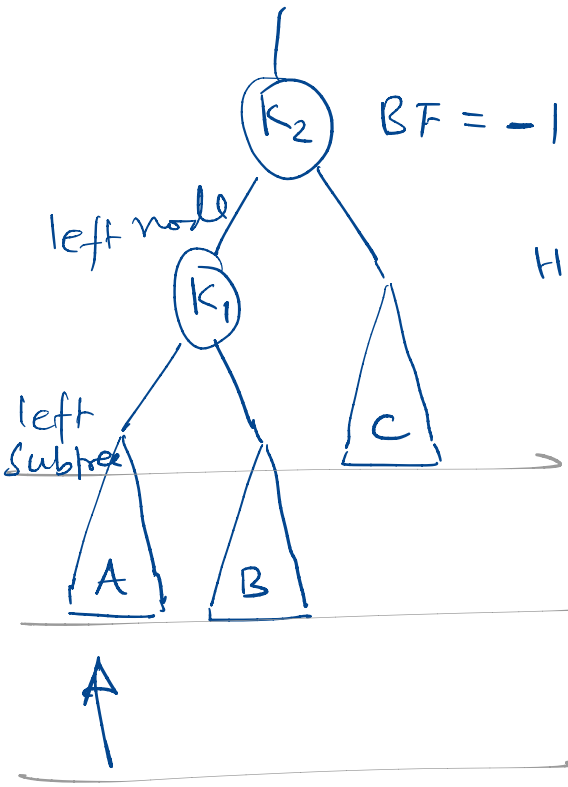
- A total of 4 cases in node insertion:

Case I : LL
Case II : LR
Case III : RL
Case IV : RR

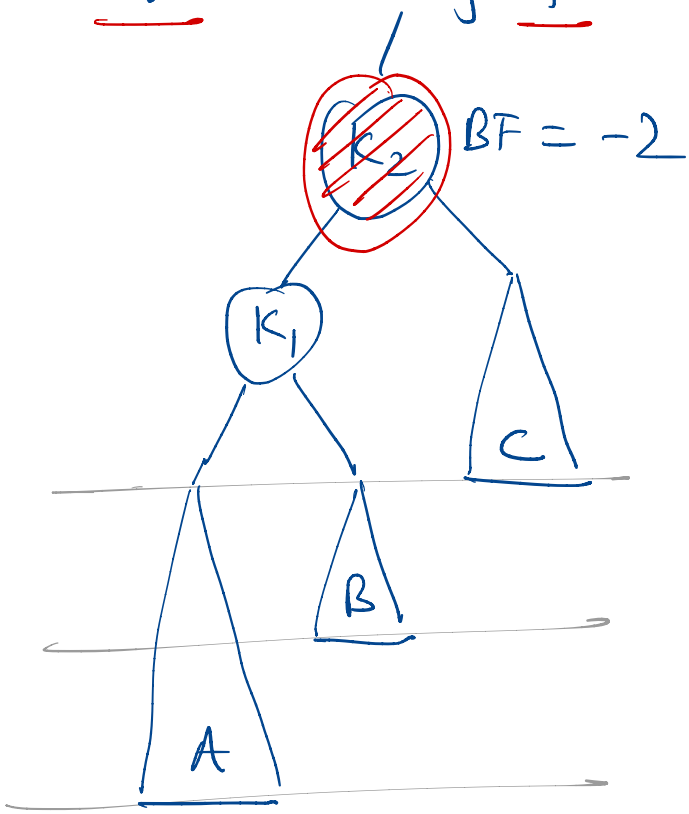


Case I : Left - Left (LL)

Insert new node in left subtree of left node

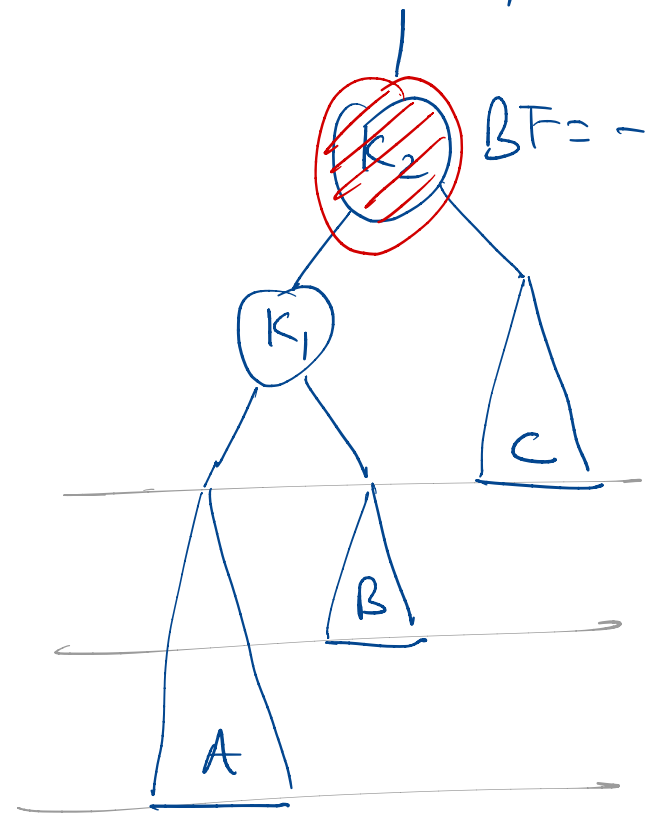


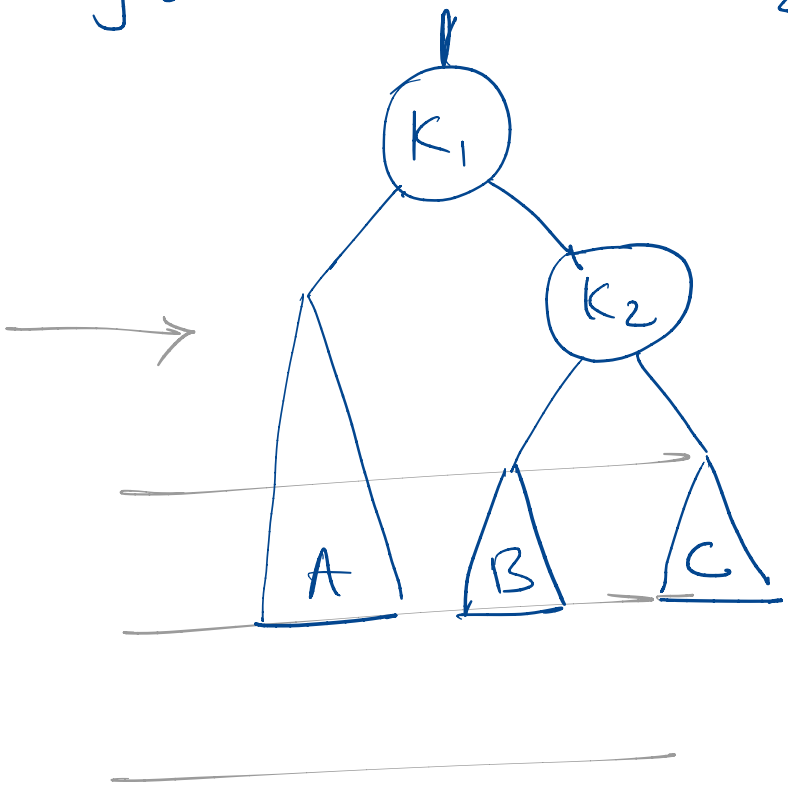
H(A) = H(B)
= H(C)



K_2 is the deepest imbalanced node

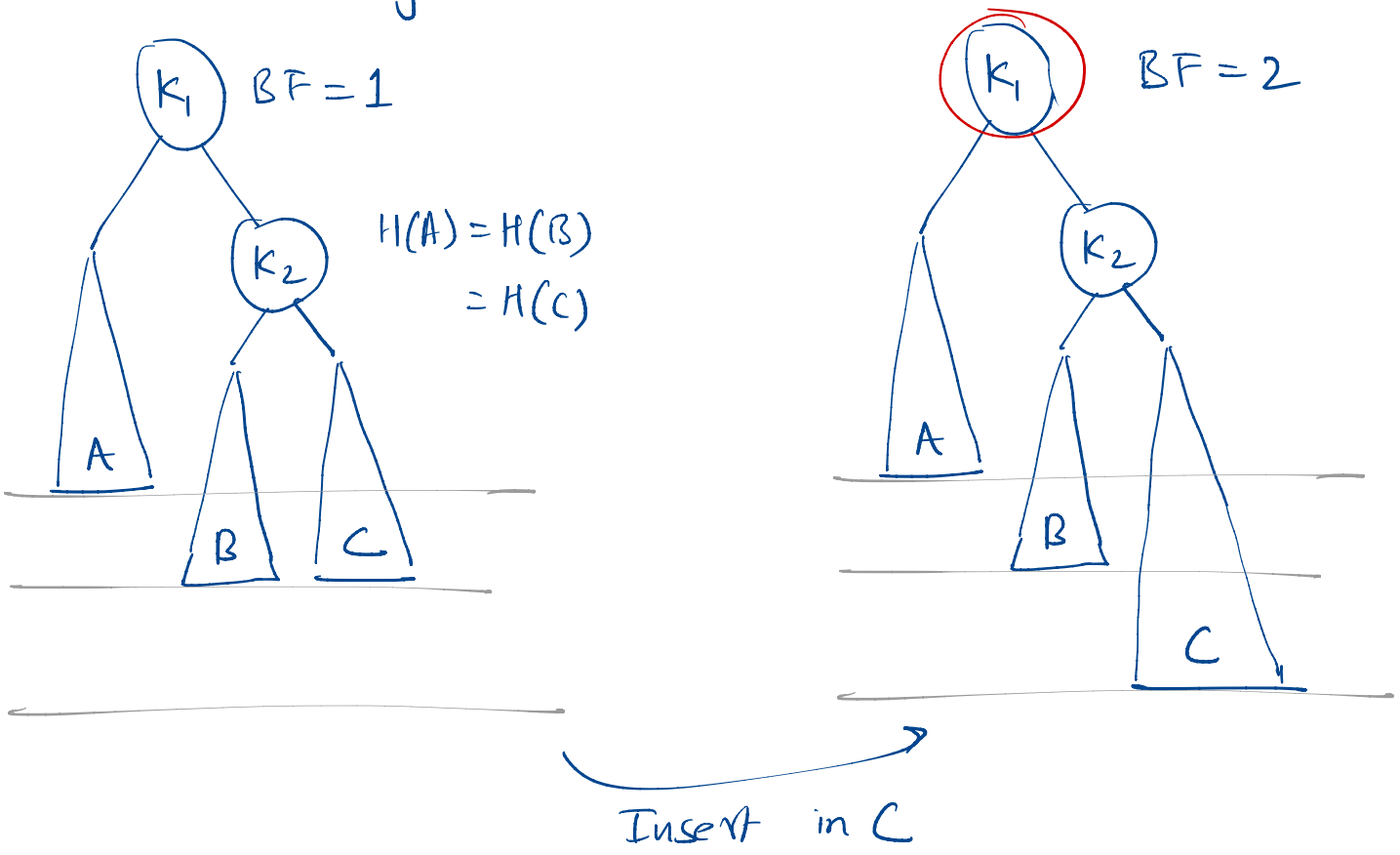
Resolution: Perform a right rotation about K_2



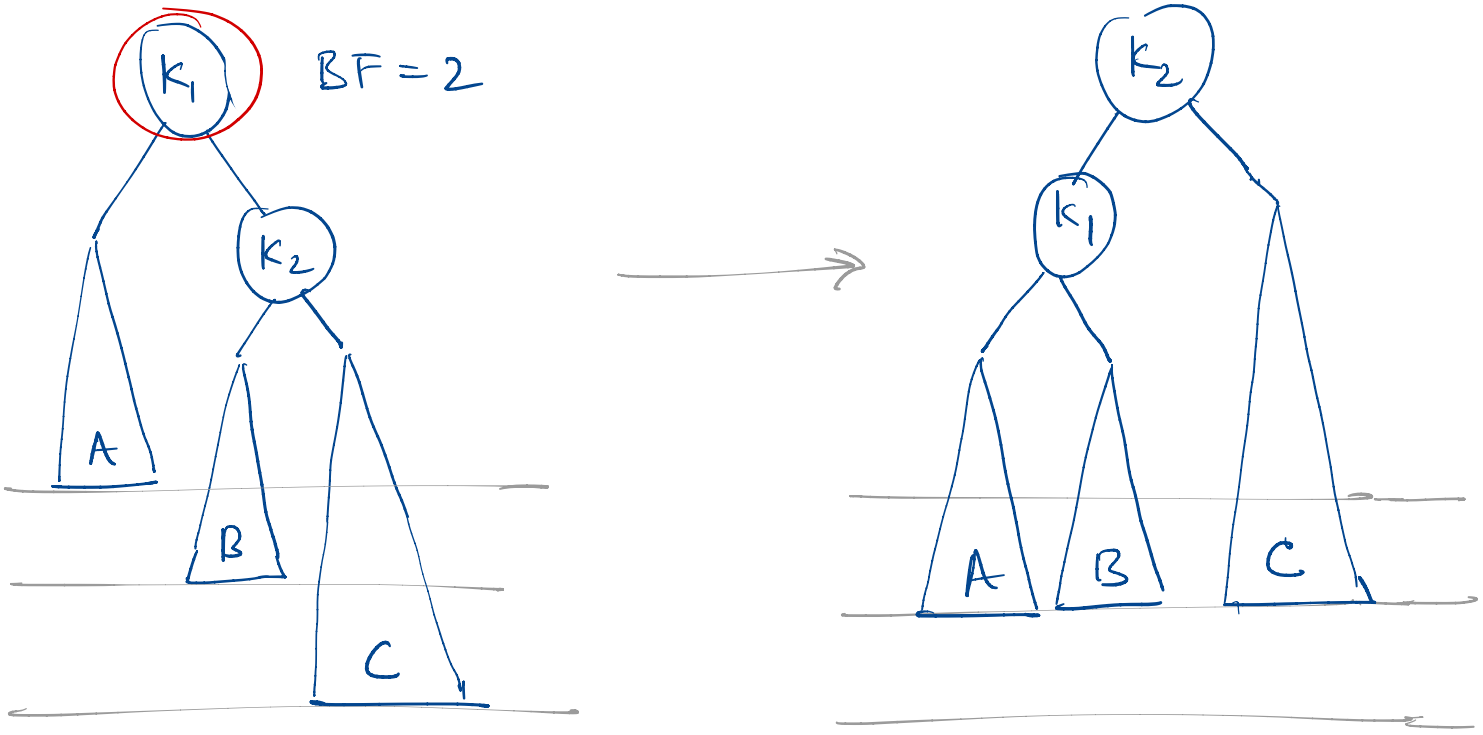


Case IV : Right-Right (RR) → Similar to LL case

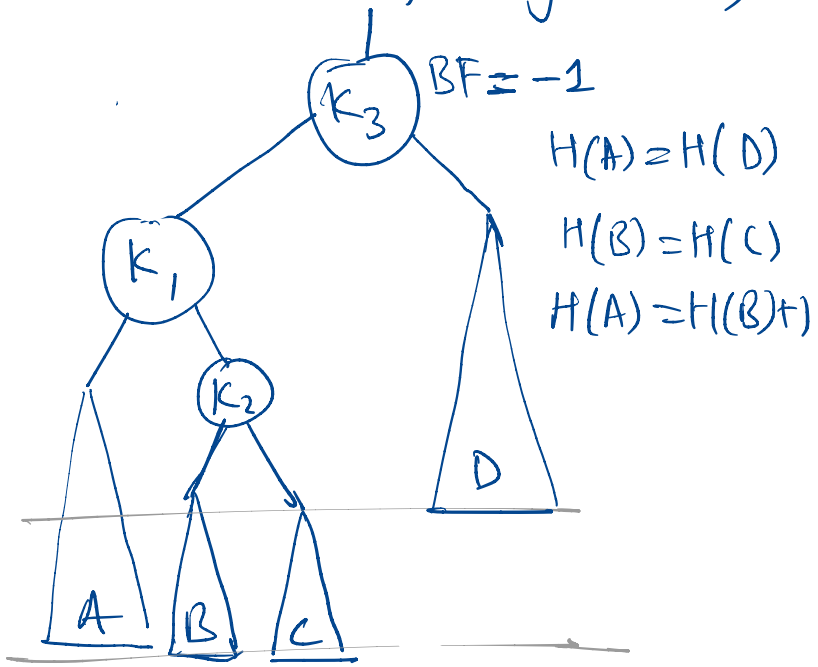
Insert new node in the right subtree of the right child



Resolution : Perform a single left rotation about K_1



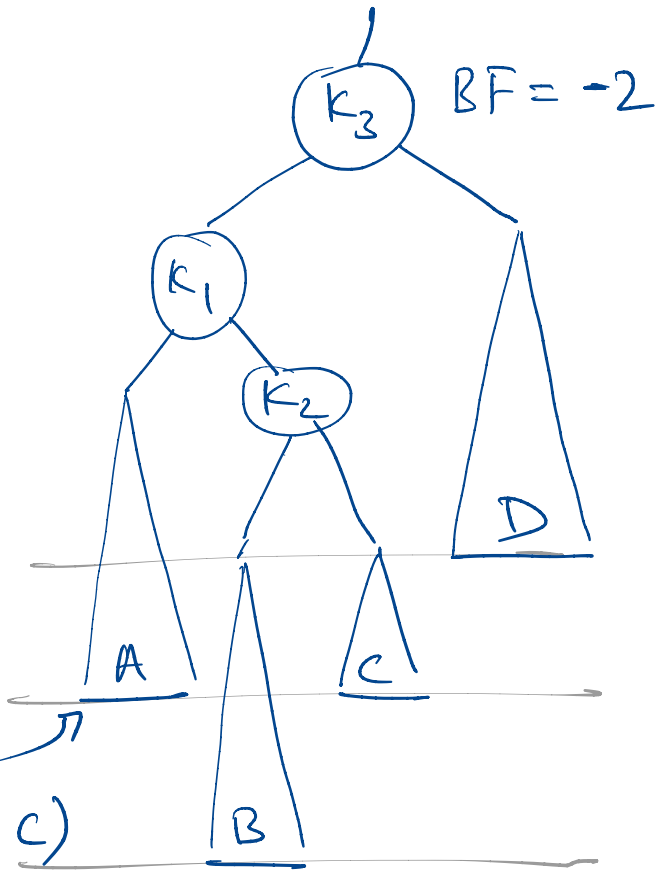
Case II : Left-Right (LR)



H(A) = H(D)

H(B) = H(C)

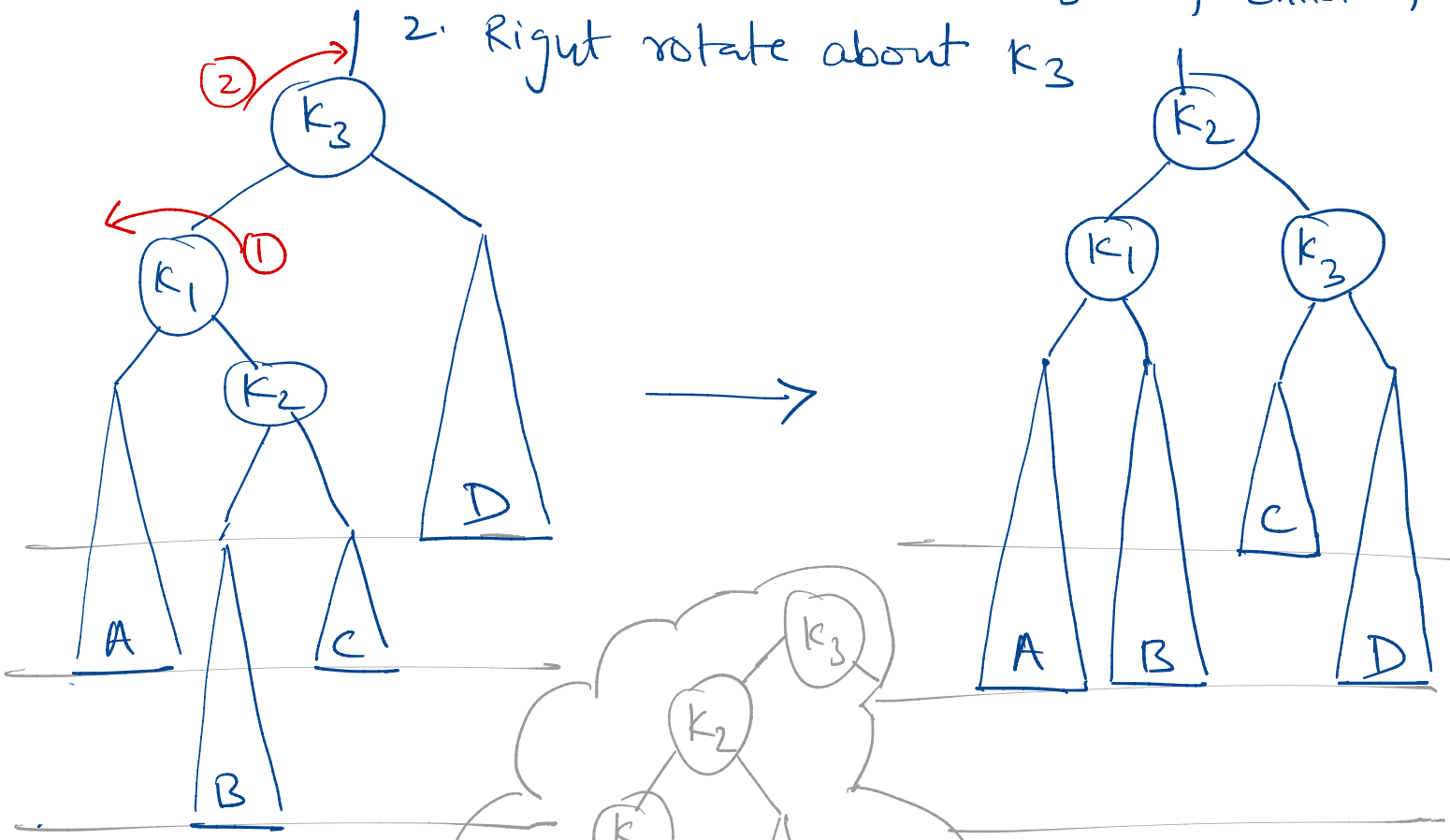
H(A) = H(B)+1

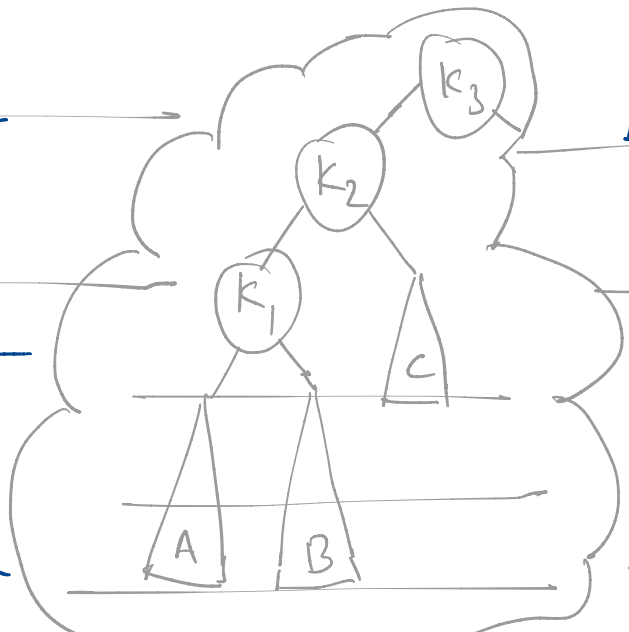


Insert in B (or C)

Resolution: 1. left rotate about K_3's left child K_1

2. Right rotate about K_3



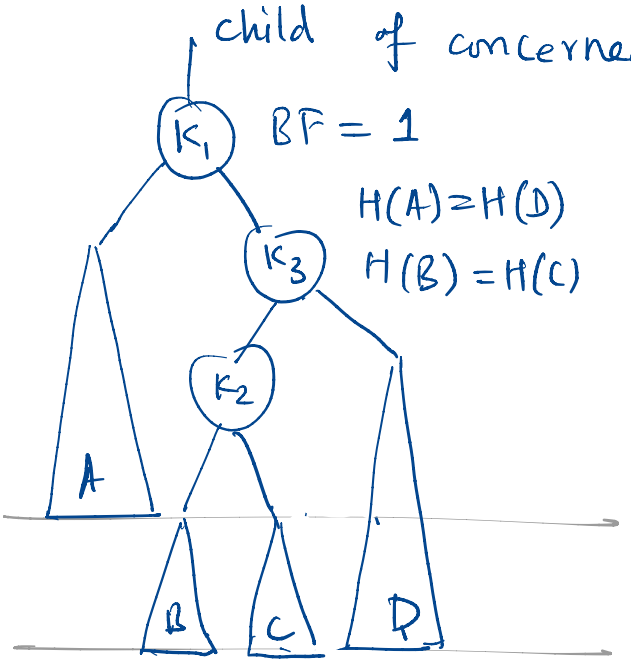


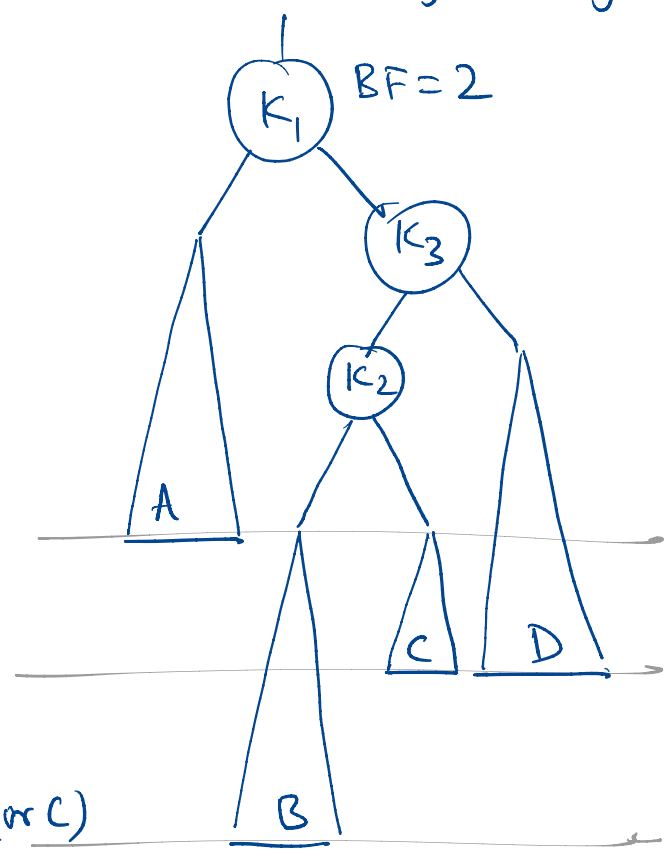
After first rotation about K_1

↳ LL case

Case III : Right-Left (RL)

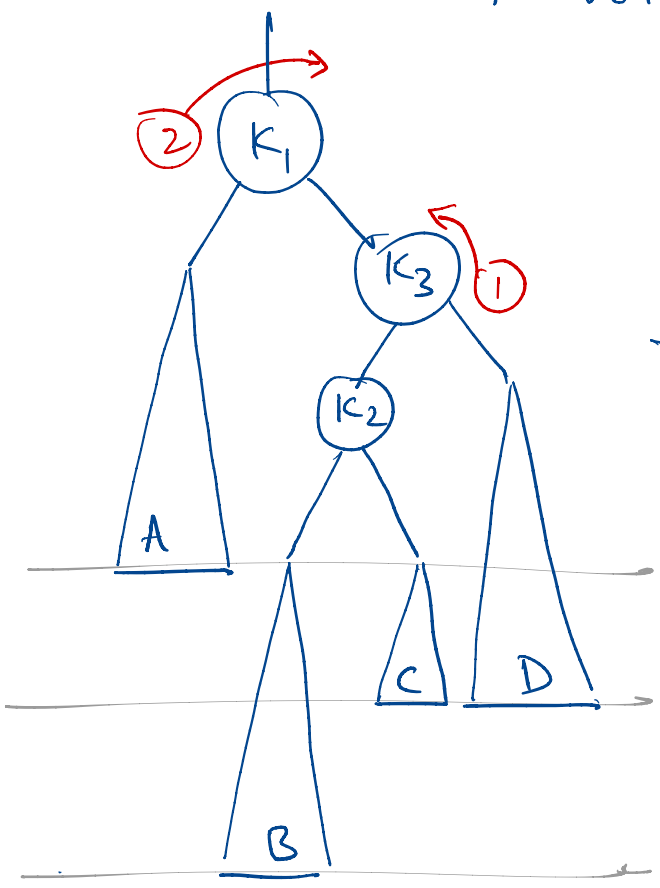
Insert a new node into the left subtree of the right child of concerned node

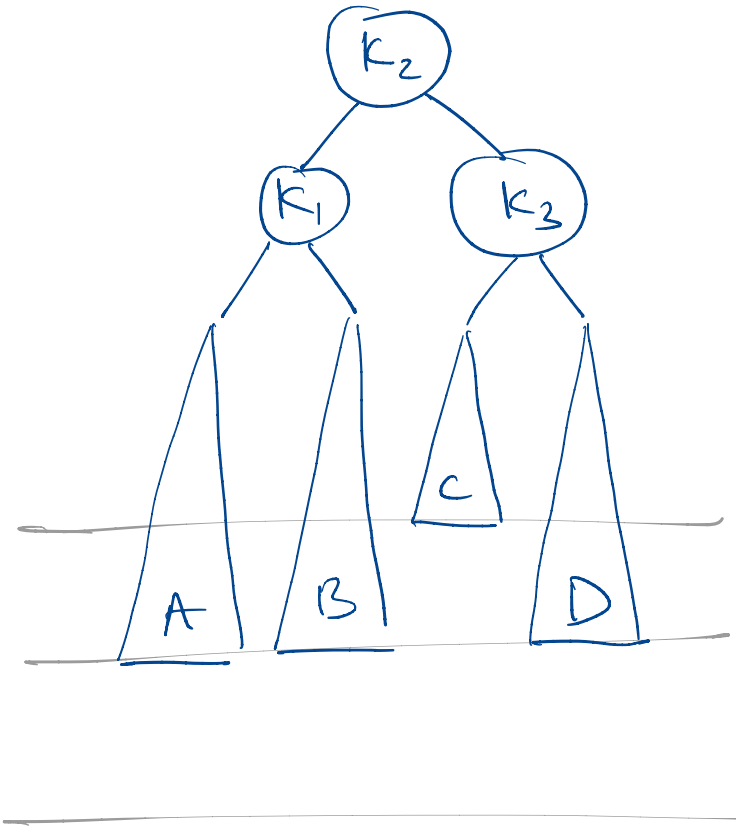




Insert in B (or C)

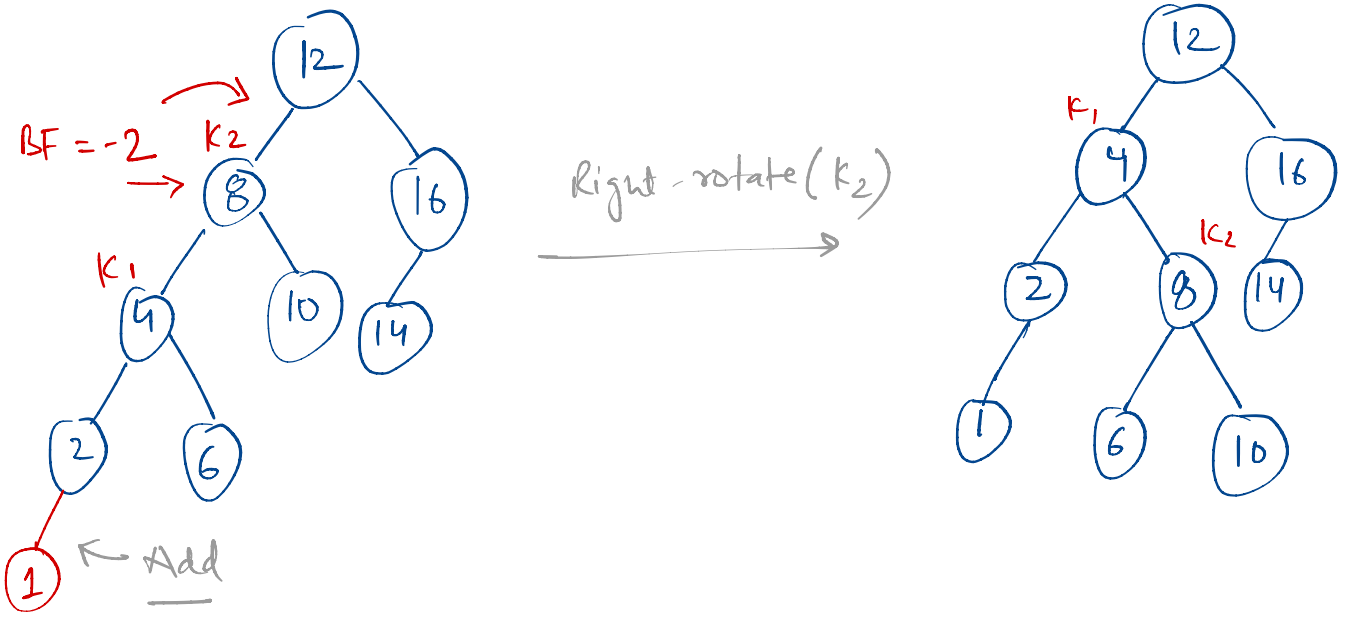
Resolution:
1. Right rotate about K_1's right child K_3
2. Left rotate about K_1



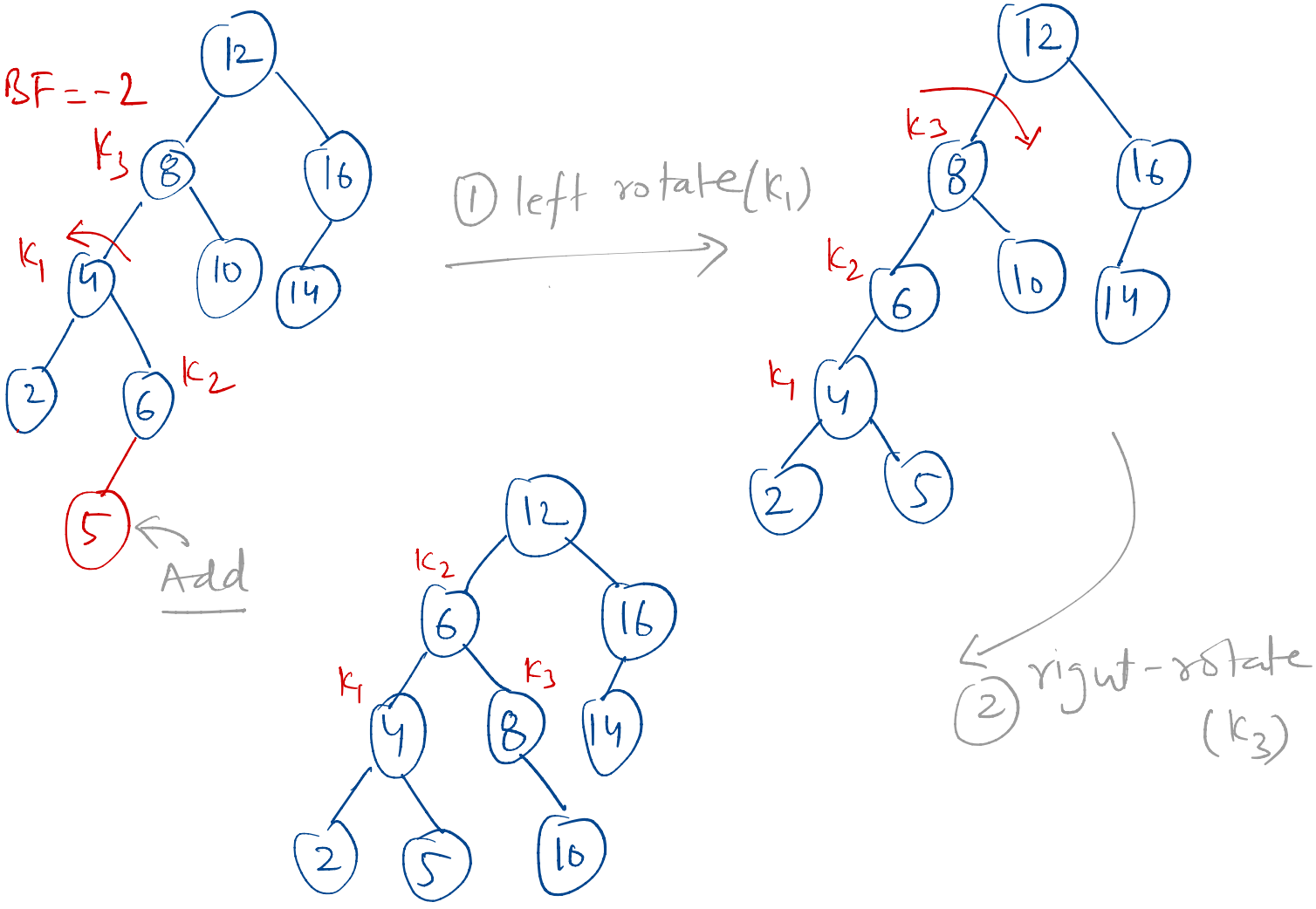


Examples:

(a) Case LL (fixed with right rotation)



(b) Case LR (fixed with left-right double rotation)



Implementing add in an AVL tree

- Perform normal BST add. When the recursive call returns, balance the node if there is an imbalance → i.e. if BF becomes −2/+2

Note: if addition was performed using iteration, then keep track of visited nodes (ancestors of the newly inserted nodes) and after insertion, fix the balance factors (leaf → root order)

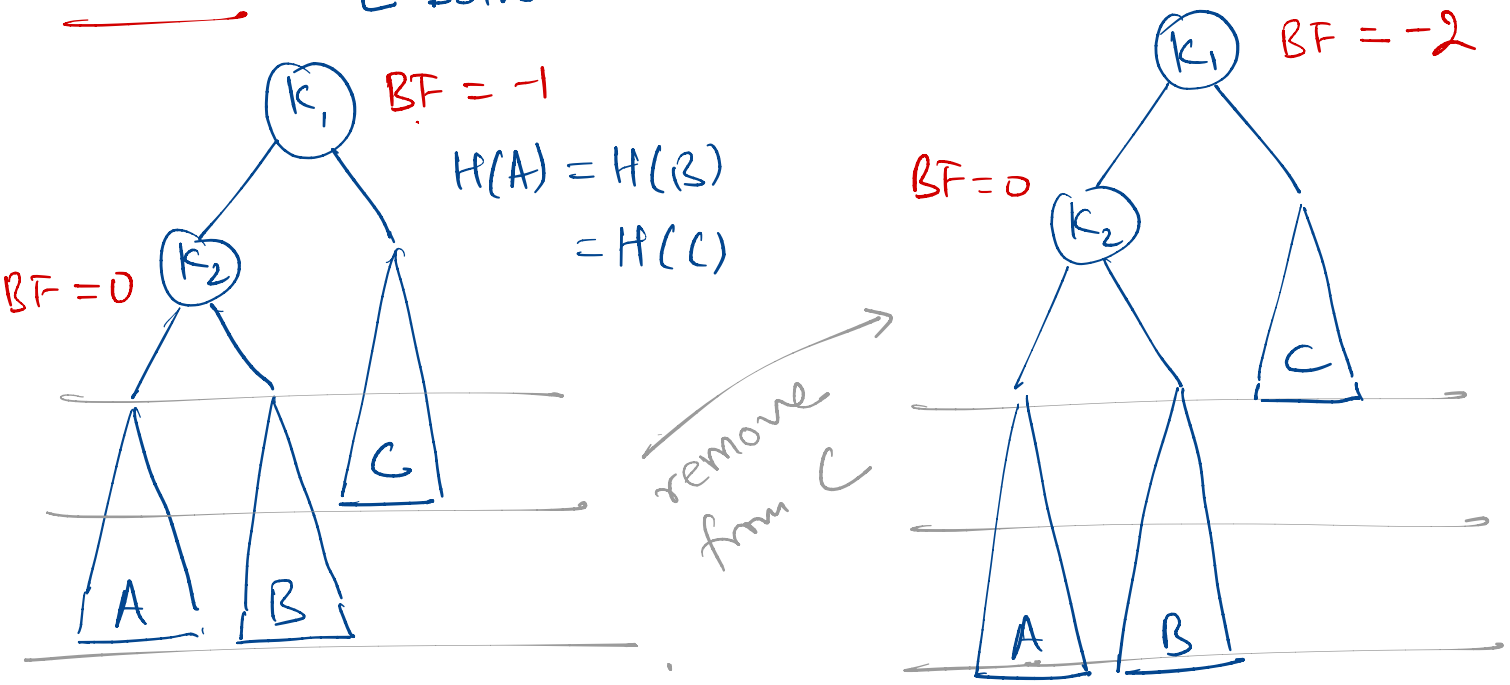
- logic to detect the case for resolution
  - Current node BF < −1 ⇒ Case LL or LR
    - ↳ Look at current node's left child BF
      - Left child BF < 0 ⇒ case LL — fix with R rotation
      - Left child BF > 0 ⇒ Case LR — fix with LR rotation
  - Current node BF > 1 ⇒ Case RR or RL
    - ↳ Look at current node's right child BF
      - Right child BF < 0 ⇒ RL Case — fix with RL rotation
      - Right child BF > 0 ⇒ RR case — fix with R rotation

AVL remove cases:

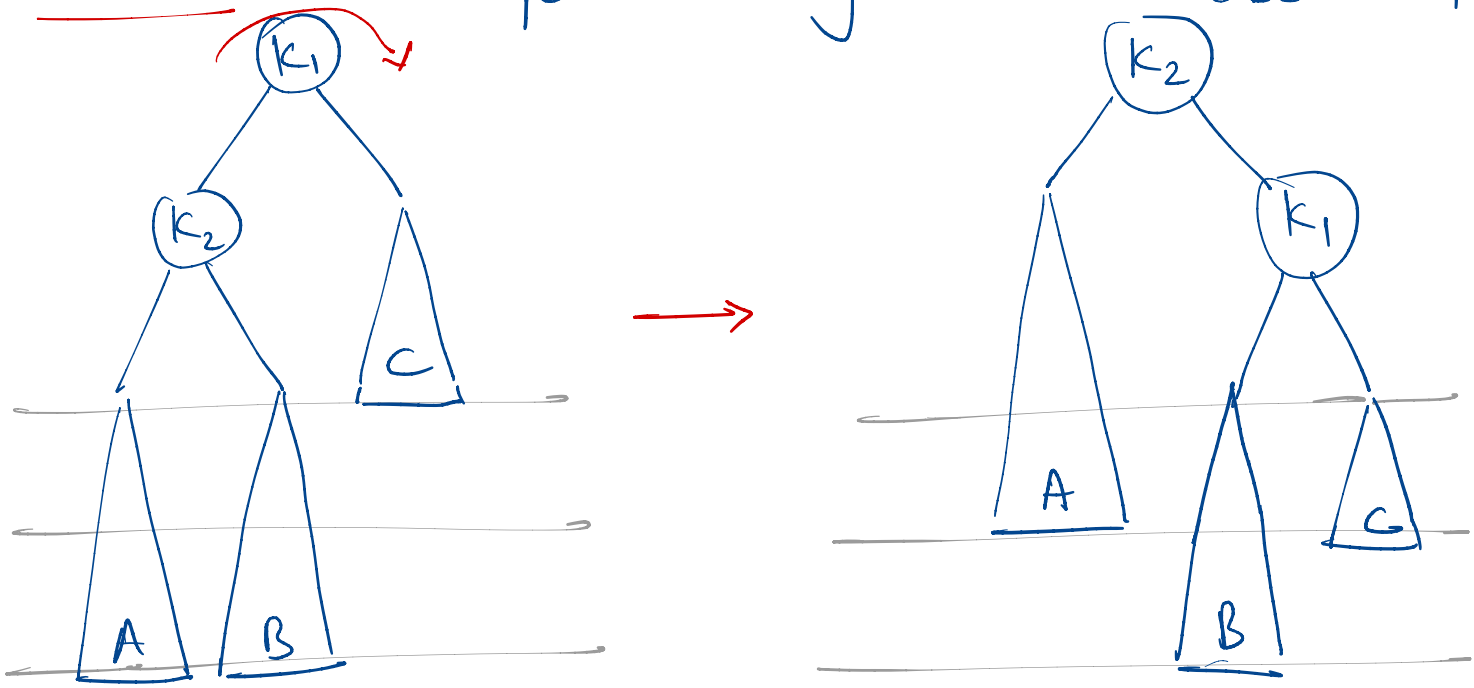
↳ Similar cases as w/ adding: LL, LR, RL, & RR

↳ Two additional cases beyond these
↳ offending subtree has a BF of 0

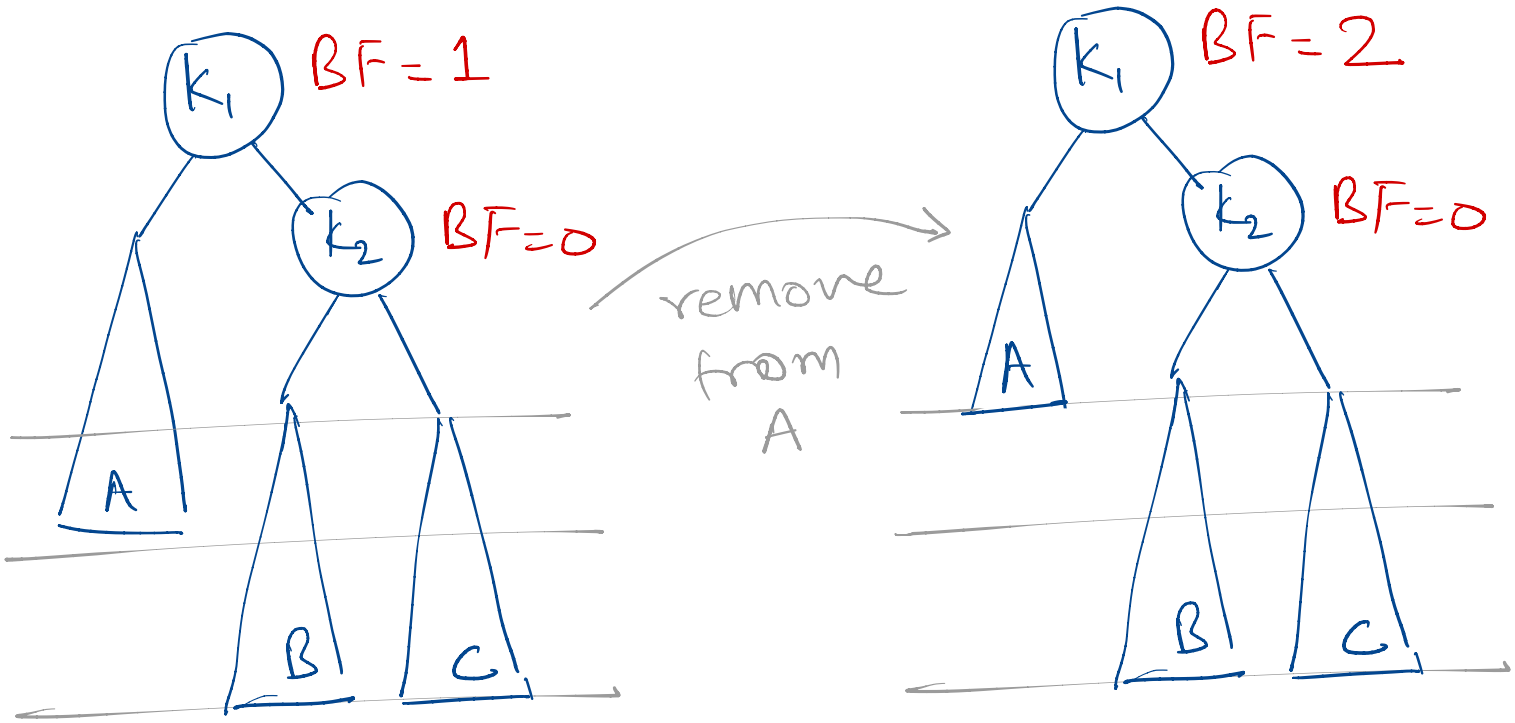
Case V: L-both



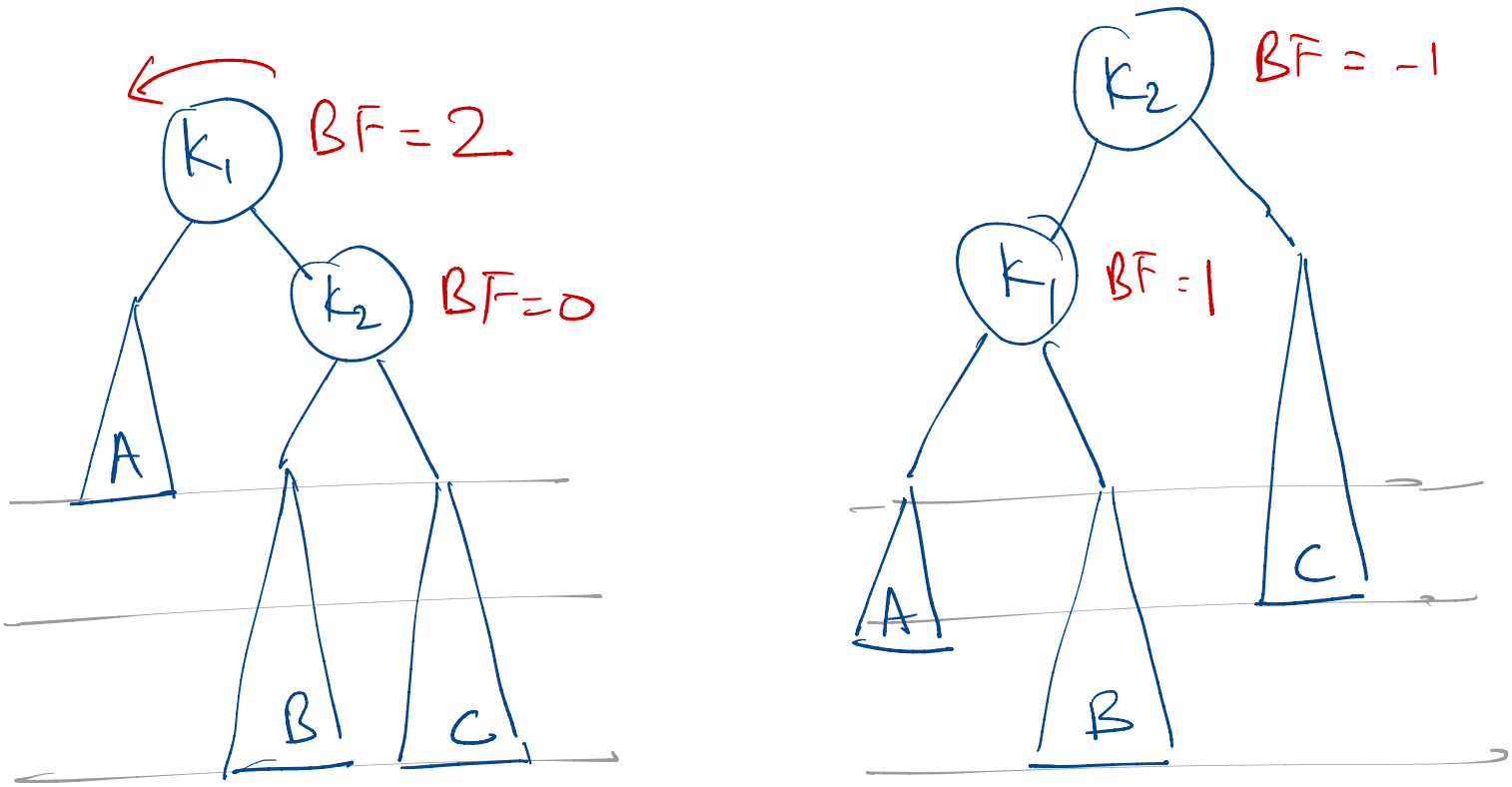
Resolution: Perform a right rotation about K_1



Case VI : R-both



Resolution: Rotate left about k_1



Implementing remove in an AVL tree:

↳ Perform normal BST removal and fix imbalance at a node while going up.

↳ Logic to detect the case of resolution:

- Current node BF < -1 ⇒ Case LL, LR, or L-both
  - ↳ Look at current node's left child BF
    - left child BF < 0 ⇒ Case LL — fix with R rotation
    - left child BF > 0 ⇒ Case LR — fix with LR rotation
    - left child BF = 0 ⇒ Case L-both — fix with R rotation

- Current node BF > 1 ⇒ Case RR, RL, or R-both
  - ↳ Look at current node's right child BF
    - right child BF < 0 ⇒ Case RL — fix with RL rotation
    - right child BF > 0 ⇒ Case RR — fix with L rotation
    - right child BF = 0 ⇒ Case R-both — fix with L rotation

Complexity: Adding $O(\lg n)$

Deleting $O(\lg n)$